CPSC-406 Report

Savana Chou
Chapman University

April 13, 2026

Abstract

Contents

1 Introduction **2**

2 Week by Week **2**

2.1 Week 1 2
2.2 Week 2 4
2.3 Week 3 10
2.4 Week 4 14
2.5 Week 5 19
2.6 Week 6 21
2.7 Week 7 24
2.8 Week 8 27

3 Synthesis **29**

4 Evidence of Participation **29**

5 Conclusion **29**

1 Introduction

2 Week by Week

2.1 Week 1

Exercise 1

1. Word acceptance table

w	accepted by A_1?	accepted by A_2?
aaa	No	Yes
aab	Yes	No
aba	No	No
abb	No	No
baa	No	Yes
bab	No	No
bba	No	No
bbb	No	No

2. Description of the languages

Language of A_1. In A_1, any word beginning with b immediately goes to a trap state. If the word begins with a, the automaton stays in state 2 while reading a's, and moves to the accepting state 3 upon reading a single b. Any further symbol leaves the accepting state.

Therefore, a word is accepted if and only if it consists of one or more a's followed by exactly one b.

$$L(A_1) = \{a^n b \mid n \geq 1\}$$

Equivalently, as a regular expression:

$$L(A_1) = a^+ b$$

Language of A_2. In A_2, the only accepting state is 3. The automaton reaches state 3 precisely after reading two consecutive a's. Any b resets the automaton to state 1.

Thus, a word is accepted if and only if it ends with at least two consecutive a's.

$$L(A_2) = \{w \in \{a, b\}^* \mid w \text{ ends with } aa\}$$

Equivalently, as a regular expression:

$$L(A_2) = (a|b)^* aa$$

Exercise 2

Alphabet: $\Sigma = \{a, b\}$.

1. DFAs for the given languages

(1) All words that end with `ab`. Let $M_1 = (Q, \Sigma, \delta, q_0, F)$ where

$$Q = \{q_0, q_1, q_2\}, \quad q_0 \text{ start}, \quad F = \{q_2\}.$$

Transitions:

	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

State q_2 represents that the word currently ends in `ab`.

(2) All words that contain `aba`. Let $M_2 = (Q, \Sigma, \delta, q_0, F)$ where

$$Q = \{q_0, q_1, q_2, q_3\}, \quad q_0 \text{ start}, \quad F = \{q_3\}.$$

Transitions:

	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_3	q_0
q_3	q_3	q_3

State q_3 means the substring `aba` has been seen.

(3) Odd number of `a`'s and odd number of `b`'s. Use four states representing parity:

$$Q = \{(E, E), (E, O), (O, E), (O, O)\}$$

(start state (E, E)). Accepting state: (O, O).

Reading a toggles the first component; reading b toggles the second component.

(4) Even number of `a`'s and odd number of `b`'s. Same construction as (3).

Start state: (E, E). Accepting state: (E, O).

Transitions again toggle parity accordingly.

(5) Any three consecutive characters contain at least one `a`. This is equivalent to saying the word does *not* contain `bbb`.

Use states counting consecutive b's:

$$Q = \{q_0, q_1, q_2, q_3\}$$

- q_0: no recent b
- q_1: one consecutive b
- q_2: two consecutive b's
- q_3: three consecutive b's (trap)

Start: q_0.

Accepting states: $\{q_0, q_1, q_2\}$.

Transitions:

- On a: go to q_0 from any state except q_3 - On b:

$$q_0 \to q_1, \quad q_1 \to q_2, \quad q_2 \to q_3, \quad q_3 \to q_3$$

(6) All words that contain `bbb`. Same structure as (5), but now:

Accepting state: $\{q_3\}$ only.

Once q_3 is reached, stay there.

2. Observations

Several patterns appear:

- Languages involving substrings (e.g. `aba`, `bbb`) use states that track partial progress toward matching the substring.
- Parity conditions use a product construction (Cartesian product) of two 2-state automata, giving 4 states.
- Conditions about avoiding a pattern (problem 5) are closely related to conditions about containing that pattern (problem 6); they differ mainly in which states are accepting.
- Many DFAs follow a systematic design pattern: track exactly the minimal information necessary to decide acceptance.

Week 1 Discord Question

Why is it important that the transition function of a DFA be total (defined for every state-symbol pair)? What would break, both formally and conceptually, if we allowed "missing" transitions?

2.2 Week 2

Exercise 1: Product Automata

Let $\Sigma = \{a, b\}$.

1. Description of $L(\mathcal{A}^{(1)})$ and $L(\mathcal{A}^{(2)})$

Language of $\mathcal{A}^{(1)}$

In $\mathcal{A}^{(1)}$, states 2 and 4 are accepting, and state 3 is a trap state with self-loops on both a and b. From the transition structure we observe:

- From state 2, reading a leads to the trap state.
- From state 4, reading b leads to the trap state.

Thus any occurrence of the substring aa or bb forces the automaton into the trap state.

Therefore,

$$L(\mathcal{A}^{(1)}) = \{w \in \{a,b\}^* \mid w \text{ contains no } aa \text{ and no } bb\}.$$

Equivalently, $L(\mathcal{A}^{(1)})$ consists of all strings whose symbols strictly alternate.

Language of $\mathcal{A}^{(2)}$

In $\mathcal{A}^{(2)}$, state 2 is the only accepting state and state 3 is a trap.

- From the start state 1, reading b leads immediately to the trap.
- Thus every accepted string must begin with a.
- The automaton alternates between states 1 and 2 on every input symbol.

Since state 2 is accepting, the automaton accepts precisely those strings that:

- start with a, and
- have odd length.

Hence,

$$L(\mathcal{A}^{(2)}) = \{w \in \{a,b\}^* \mid w \text{ starts with } a \text{ and } |w| \text{ is odd}\}.$$

2. Construction of the Intersection Automaton

The product automaton has state set

$$Q = Q^{(1)} \times Q^{(2)} = \{1,2,3,4\} \times \{1,2,3\}, \quad \text{start state } (1,1),$$

with transition function defined by

$$\delta\big((i,j),x\big) = \big(\delta_1(i,x), \delta_2(j,x)\big).$$

Reachable states from $(1,1)$ and their transitions:

State	on a	on b
$(1,1)$	$(2,2)$	$(4,3)$
$(2,2)$	$(3,1)$	$(4,1)$
$(4,3)$	$(3,3)$	$(2,3)$
$(3,1)$	$(3,2)$	$(3,3)$
$(4,1)$	$(3,2)$	$(2,3)$
$(3,3)$	$(3,3)$	$(3,3)$
$(2,3)$	$(3,3)$	$(4,3)$
$(3,2)$	$(3,1)$	$(3,1)$

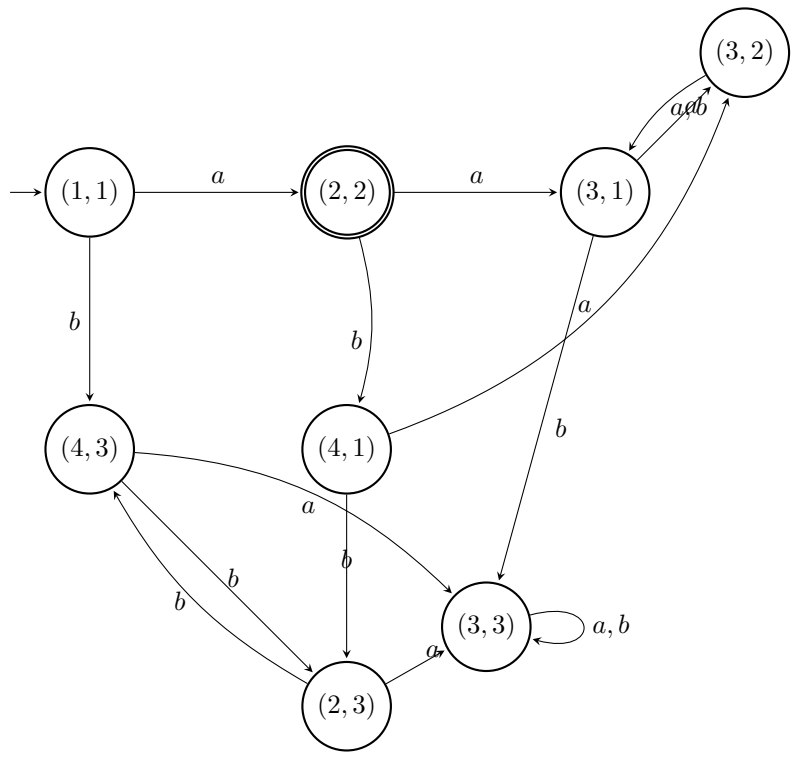
Thus, the reachable states are

$$\{(1,1),(2,2),(4,3),(3,1),(4,1),(3,3),(2,3),(3,2)\}.$$

The accepting states of the intersection automaton are

$$F = F^{(1)} \times F^{(2)} = \{2,4\} \times \{2\} = \{(2,2),(4,2)\}.$$

Since $(4,2)$ is unreachable, the only accepting state is $(2,2)$.



3. Correctness of the Product Construction

By definition of the product construction, a string w is accepted by $\mathcal{A}$ if and only if:

- the run of w in $\mathcal{A}^{(1)}$ ends in an accepting state, and
- the run of w in $\mathcal{A}^{(2)}$ ends in an accepting state.

Thus,

$$L(\mathcal{A}) = L(\mathcal{A}^{(1)}) \cap L(\mathcal{A}^{(2)}).$$

4. Construction for the Union

To obtain an automaton $\mathcal{A}'$ such that

$$L(\mathcal{A}') = L(\mathcal{A}^{(1)}) \cup L(\mathcal{A}^{(2)}),$$

we keep:

- the same state set $Q_1 \times Q_2$,
- the same start state $(1, 1)$,
- the same transition function.

We change only the accepting states to:

$$F' = (F_1 \times Q_2) \cup (Q_1 \times F_2).$$

That is, a state (p, q) is accepting if either

$$p \in F_1 \quad \text{or} \quad q \in F_2.$$

Exercise 2: More Automata

Let $\Sigma = \{a, b\}$.

1. Description of the Languages

Language of $\mathcal{B}^{(1)}$

The automaton $\mathcal{B}^{(1)}$ has states $\{p_0, p_1, p_2\}$ where p_0 is the start and only accepting state. Each b leaves the state unchanged, while each a moves cyclically:

$$p_0 \xrightarrow{a} p_1 \xrightarrow{a} p_2 \xrightarrow{a} p_0.$$

Thus the automaton counts the number of a's modulo 3. Since only p_0 is accepting, we obtain

$$L(\mathcal{B}^{(1)}) = \{\, w \in \{a, b\}^* \mid \#_a(w) \equiv 0 \pmod 3 \,\}.$$

In words: all strings whose number of a's is divisible by 3.

Language of $\mathcal{B}^{(2)}$

The automaton $\mathcal{B}^{(2)}$ has states $\{q_0, q_1, q_2\}$ where q_0 is the start and only accepting state, and q_2 is a trap state with loops on both a and b.

The transitions show:

$$q_0 \xrightarrow{a} q_1, \qquad q_1 \xrightarrow{a} q_2.$$

Thus any occurrence of two consecutive a's sends the automaton to the trap state q_2. Therefore,

$$L(\mathcal{B}^{(2)}) = \{\, w \in \{a, b\}^* \mid w \text{ does not contain } aa \,\}.$$

2. Construction of the Intersection Automaton

Define the state set as the Cartesian product:

$$Q = \{p_0, p_1, p_2\} \times \{q_0, q_1, q_2\}, \quad \text{with start state } (p_0, q_0).$$

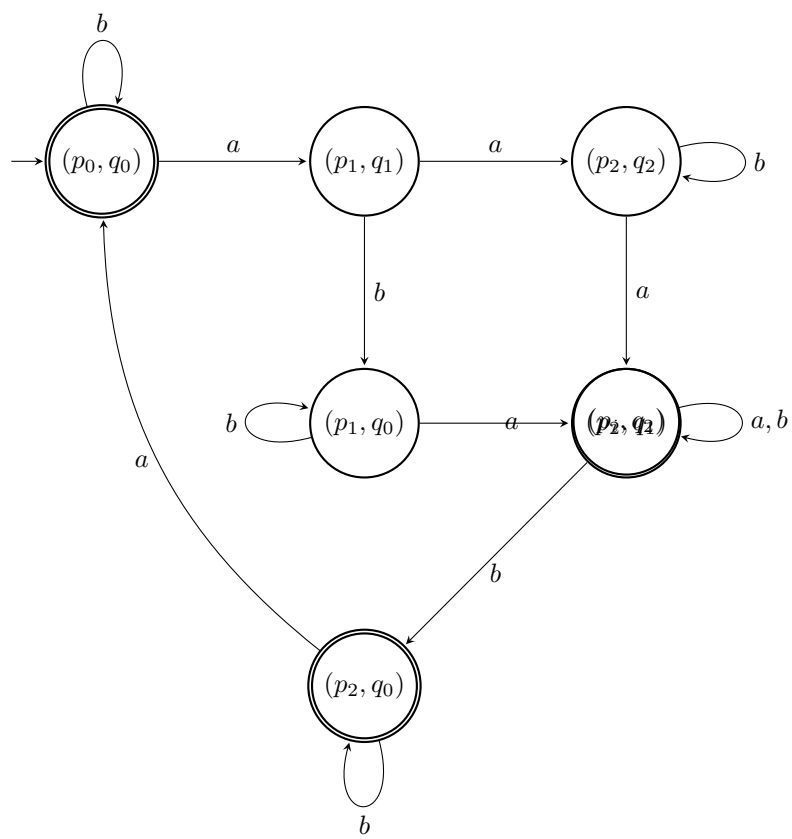
We compute the reachable states and transitions:

State	input a	input b
(p_0, q_0)	(p_1, q_1)	(p_0, q_0)
(p_1, q_1)	(p_2, q_2)	(p_1, q_0)
(p_2, q_2)	(p_0, q_2)	(p_2, q_2)
(p_1, q_0)	(p_2, q_1)	(p_1, q_0)
(p_0, q_2)	(p_1, q_2)	(p_0, q_2)
(p_2, q_1)	(p_0, q_2)	(p_2, q_0)
(p_1, q_2)	(p_2, q_2)	(p_1, q_2)
(p_2, q_0)	(p_0, q_1)	(p_2, q_0)
(p_0, q_1)	(p_1, q_2)	(p_0, q_0)

The accepting states of the intersection automaton are the product of the accepting states of the original DFAs:

$$F = F^{(1)} \times F^{(2)} = \{p_0, p_2\} \times \{q_0\} = \{(p_0, q_0), (p_2, q_0)\}.$$

Among the reachable states, only (p_0, q_0) and (p_2, q_0) satisfy this condition.



Note: The states (p_0, q_2), (p_1, q_2), and (p_2, q_2) are merged into a single trap state for simplicity.

Part 3: Verification. By construction, a string w is accepted by the intersection automaton $\mathcal{B}$ if and only if it is accepted by both original automata, i.e.,

$$w \in L(\mathcal{B}) \iff w \in L(\mathcal{B}^{(1)}) \text{ and } w \in L(\mathcal{B}^{(2)}).$$

Hence, the language recognized by $\mathcal{B}$ is

$$L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)}) = \{w \mid \#_a(w) \not\equiv 1 \pmod 3\} \cap \{w \mid w \text{ contains no } aa\}.$$

3. Correctness of the Construction

By definition of the product construction, a string w is accepted by $\mathcal{B}$ if and only if:

- the run of w in $\mathcal{B}^{(1)}$ ends in p_0, and
- the run of w in $\mathcal{B}^{(2)}$ ends in q_0.

Hence,

$$L(\mathcal{B}) = L(\mathcal{B}^{(1)}) \cap L(\mathcal{B}^{(2)}).$$

4. Construction of $\mathcal{B}'$ for the Union with a Complement

We want an automaton $\mathcal{B}'$ such that

$$L(\mathcal{B}') = L(\mathcal{B}^{(1)}) \cup \overline{L(\mathcal{B}^{(2)})}.$$

Step 1: Complement $\mathcal{B}^{(2)}$. Since $\mathcal{B}^{(2)}$ is a complete DFA, we obtain $\overline{\mathcal{B}^{(2)}}$ by swapping accepting and non-accepting states. Thus q_1 and q_2 become accepting, and q_0 becomes rejecting.

Step 2: Product Construction for the Union. We keep:

- the same state set $Q_1 \times Q_2$,
- the same start state (p_0, q_0),
- the same transition function.

We change the accepting states to:

$$F' = (F_1 \times Q_2) \cup (Q_1 \times F_{\overline{2}}).$$

That is, a state (p, q) is accepting if either

$$p = p_0 \quad \text{or} \quad q \in \{q_1, q_2\}.$$

Exercise 2.2.7 — State with Self-Loops

Problem. Consider a DFA $\mathcal{A}$ and a state q such that for every input symbol $a \in \Sigma$, the transition function satisfies $\delta(q, a) = q$. Prove, using induction on the length of input strings, that for any string $w \in \Sigma^*$, the extended transition function satisfies $\hat{\delta}(q, w) = q$.

Solution. We proceed by induction on the length of the string w, denoted $|w|$.

Base case: $|w| = 0$, i.e., $w = \varepsilon$ (the empty string).

By the definition of the extended transition function:

$$\hat{\delta}(q, \varepsilon) = q.$$

Hence, the statement is true for the base case.

Inductive step:

Inductive hypothesis (IH): Assume that for all strings w' of length n, we have $\hat{\delta}(q, w') = q$.

Goal: Show that for any string w of length $n+1$, $\hat{\delta}(q, w) = q$.

Let w be a string of length $n+1$, and write $w = w'a$ where $|w'| = n$ and $a \in \Sigma$.

By the recursive definition of $\hat{\delta}$:

$$\hat{\delta}(q, w) = \hat{\delta}(q, w'a) = \delta\big(\hat{\delta}(q, w'), a\big).$$

Applying the inductive hypothesis, $\hat{\delta}(q, w') = q$, so:

$$\delta\big(\hat{\delta}(q, w'), a\big) = \delta(q, a).$$

Since q has a self-loop for every symbol, $\delta(q, a) = q$ for all $a \in \Sigma$. Therefore:

$$\hat{\delta}(q, w) = q.$$

This completes the inductive step, and by induction, the result holds for all $w \in \Sigma^*$.

Week 2 Discord Question

In both exercises, we constructed product automata to recognize intersections and modified acceptance conditions to recognize unions or complements. Suppose instead we were given only one of the product automata (without being told it was constructed from two smaller DFAs). How could we determine whether this automaton can be decomposed into a product of two smaller DFAs?

2.3 Week 3

Homework 1

Consider the NFA $\mathcal{A}$ over the alphabet $\Sigma = \{a, b, c\}$.

Part 1: Description of the language. By inspecting the diagram, we see that $\mathcal{A}$ accepts strings that:

- Begin with any combination of a's and b's (remaining in state 0),
- Followed by exactly one a (moving to state 1),
- And end with either a b (reaching accepting state 2) or a c (reaching accepting state 3).

$$\boxed{L(\mathcal{A}) = (a \mid b)^* a (b \mid c)}$$

Part 2: Formal definition of the NFA. The automaton is defined as $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ with:

- $Q = \{0, 1, 2, 3\}$
- $\Sigma = \{a, b, c\}$
- Start state $q_0 = 0$
- Accepting states $F = \{2, 3\}$
- Transition function $\delta : Q \times \Sigma \to \mathcal{P}(Q)$ given by:

δ	a	b	c
0	$\{0, 1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\{3\}$
2	$\emptyset$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

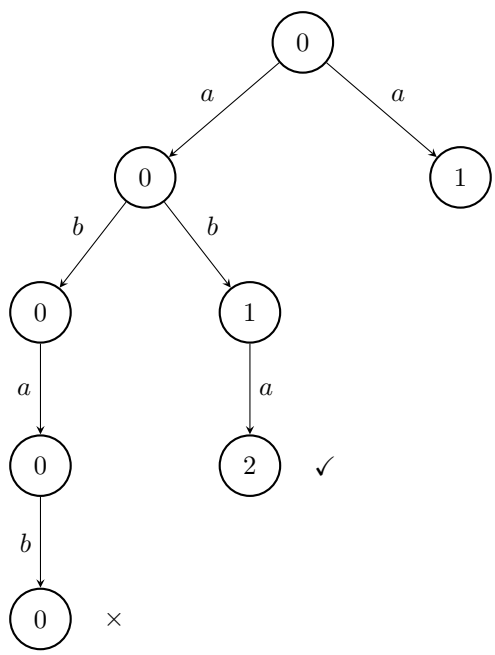
Part 3: Tracing all paths for $w = abab$. We examine all possible transitions through $\mathcal{A}$ for the input string $abab$:

Path 1: $0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 0 \xrightarrow{b} 0$ (ends in state 0, **not accepting**)

Path 2: $0 \xrightarrow{a} 0 \xrightarrow{b} 0 \xrightarrow{a} 1 \xrightarrow{b} 2$ (ends in state 2, **ACCEPTING**)

Path 3: $0 \xrightarrow{a} 1 \xrightarrow{b} 2$ (cannot continue for remaining symbols)

Visual representation of successful paths:



Since at least one path reaches an accepting state, $w = abab$ is **accepted**.

Part 4: Determinization via the power set construction. The DFA $\mathcal{A}^D = (Q^D, \Sigma, \delta^D, q_0^D, F^D)$ is defined as:

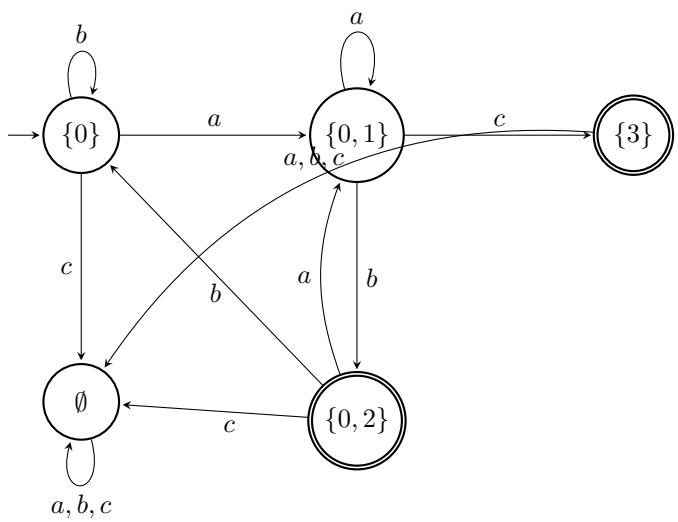
- $Q^D = \mathcal{P}(Q)$ (we list only reachable subsets)

- $q_0^D = \{0\}$
- $F^D = \{S \subseteq Q \mid S \cap \{2,3\} \neq \emptyset\}$
- $\delta^D(S,x) = \bigcup_{q \in S} \delta(q,x)$

Transition table for $\mathcal{A}^D$:

State	a	b	c
$\rightarrow \{0\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$\{0,1\}$	$\{0,1\}$	$\{0,2\}$	$\{3\}$
$*\{0,2\}$	$\{0,1\}$	$\{0\}$	$\emptyset$
$*\{3\}$	$\emptyset$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

Transition diagram for $\mathcal{A}^D$:



The determinized automaton has **5 reachable states**.

Homework 2

Consider the NFA $\mathcal{A}$ over $\Sigma = \{0,1\}$.

Formal specification. $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ with:

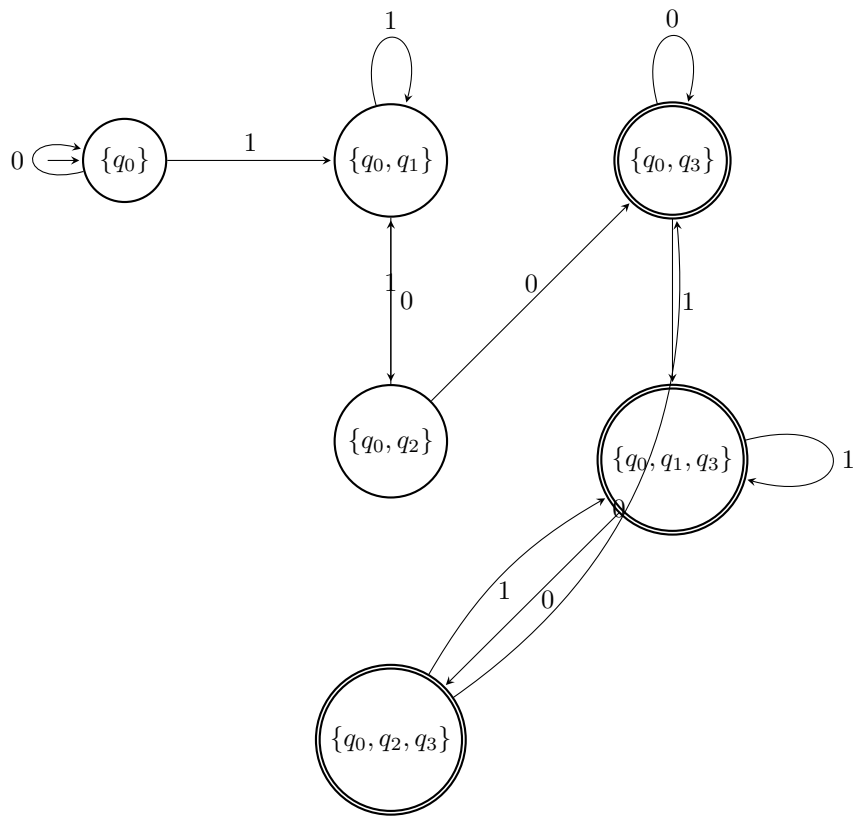
- $Q = \{q_0, q_1, q_2, q_3\}$
- Start state q_0
- $F = \{q_3\}$
- Transition function δ as follows:

δ	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\{q_1\}$
q_3	$\{q_3\}$	$\{q_3\}$

Part 1: Determinization (power set construction). Starting from $\{q_0\}$, the reachable subsets are computed:

State	0	1
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1\}$
$\{q_0, q_2\}$	$\{q_0, q_3\}$	$\{q_0, q_1\}$
$*\{q_0, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$
$*\{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\}$
$*\{q_0, q_2, q_3\}$	$\{q_0, q_3\}$	$\{q_0, q_1, q_3\}$

Transition diagram for the determinized DFA:

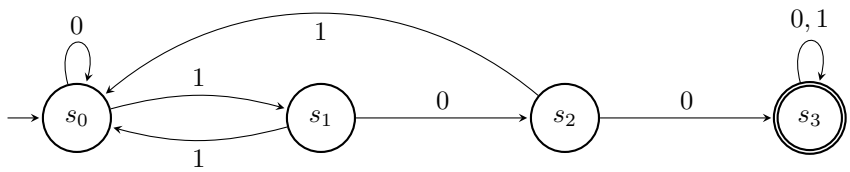


The determinized DFA has **6 reachable states**.

Part 2: Minimal DFA. Yes, a smaller DFA exists for the same language. The language consists of all strings containing the substring 100:

$$L(\mathcal{A}) = (0 \mid 1)^* 100 (0 \mid 1)^*$$

Minimal DFA with 4 states:



State meanings:

- s_0: no match for 100 so far
- s_1: most recently saw 1
- s_2: most recently saw 10
- s_3: matched 100 (accepting, remains accepting)

This minimal DFA reduces the number of states from **6** to **4** by tracking progress toward the target substring.

Week 3 Discord Question

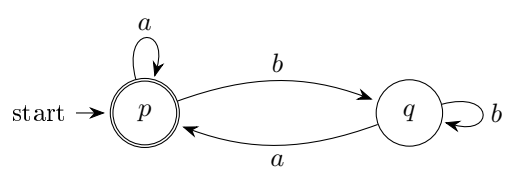
Is there a structural way to predict which subsets will collapse during minimization before fully constructing the DFA? In other words, can we detect equivalent subset-states early from properties of the original NFA?

2.4 Week 4

The DFA $\mathcal{A}$

- **States:** $p = 1$, $q = 2$
- **Start state:** p (also an accept state, indicated by double circle)
- **Accept states:** $\{p\}$
- **Transitions:**

State	a	b
$\rightarrow * p$	p	q
q	p	q



Kleene's Algorithm

Setup

Kleene's algorithm computes regular expressions $R_{ij}^{(k)}$ representing the set of all strings that take state i to state j using only intermediate states from $\{1, \ldots, k\}$.

The recurrence is:

$$R_{ij}^{(k)} = R_{ik}^{(k-1)} \left(R_{kk}^{(k-1)}\right)^* R_{kj}^{(k-1)} \; + \; R_{ij}^{(k-1)}$$

with base case (no intermediate states, $k = 0$):

$$R_{ij}^{(0)} = \begin{cases} a_1 + a_2 + \cdots + a_m & \text{if there are transitions } i \xrightarrow{a_\ell} j \text{ (and } i \neq j) \\ a_1 + \cdots + a_m + \varepsilon & \text{if } i = j \text{ and there are transitions } i \xrightarrow{a_\ell} i \\ \varepsilon & \text{if } i = j \text{ and no self-loop} \\ \emptyset & \text{if no transition from } i \text{ to } j \end{cases}$$

Step 0: Base Cases $R_{ij}^{(0)}$

Reading directly from the transition table (states $1 = p$, $2 = q$):

$$R_{11}^{(0)} = a + \varepsilon = a^* \quad \text{(self-loop on } a\text{, plus } \varepsilon \text{ for staying)}$$

$$R_{12}^{(0)} = b \quad \text{(transition } p \xrightarrow{b} q\text{)}$$

$$R_{21}^{(0)} = a \quad \text{(transition } q \xrightarrow{a} p\text{)}$$

$$R_{22}^{(0)} = b + \varepsilon = b^* \quad \text{(self-loop on } b\text{, plus } \varepsilon\text{)}$$

Note: Since $R_{ii}^{(0)}$ is used inside a Kleene star later, we keep them as $a+\varepsilon$ and $b+\varepsilon$ here for clarity, but note $(a+\varepsilon)^* = a^*$ and $(b+\varepsilon)^* = b^*$.

Step 1: $R_{ij}^{(1)}$ — Intermediate state allowed: $\{1\}$

Using the recurrence with $k = 1$:

$$R_{ij}^{(1)} = R_{i1}^{(0)} \left(R_{11}^{(0)}\right)^* R_{1j}^{(0)} + R_{ij}^{(0)}$$

Note $(R_{11}^{(0)})^* = (a+\varepsilon)^* = a^*$.

$\boldsymbol{R_{11}^{(1)}}$:

$$R_{11}^{(1)} = R_{11}^{(0)}(R_{11}^{(0)})^* R_{11}^{(0)} + R_{11}^{(0)} = (a+\varepsilon) \cdot a^* \cdot (a+\varepsilon) + (a+\varepsilon) = a^* + (a+\varepsilon) = a^*$$

$$\boxed{R_{11}^{(1)} = a^*}$$

$\boldsymbol{R_{12}^{(1)}}$:

$$R_{12}^{(1)} = R_{11}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)} + R_{12}^{(0)} = (a+\varepsilon) \cdot a^* \cdot b + b = a^*b + b = a^*b$$

$$\boxed{R_{12}^{(1)} = a^*b}$$

$\boldsymbol{R_{21}^{(1)}}$:

$$R_{21}^{(1)} = R_{21}^{(0)}(R_{11}^{(0)})^* R_{11}^{(0)} + R_{21}^{(0)} = a \cdot a^* \cdot (a+\varepsilon) + a = a \cdot a^* + a = a^+$$

Since $a \cdot a^* = a^+$ and $a^+ + a = a^+$:

$$\boxed{R_{21}^{(1)} = a^+}$$

$\boldsymbol{R_{22}^{(1)}}$:

$$R_{22}^{(1)} = R_{21}^{(0)}(R_{11}^{(0)})^* R_{12}^{(0)} + R_{22}^{(0)} = a \cdot a^* \cdot b + (b + \varepsilon) = a^+ b + b + \varepsilon = a^* b + \varepsilon$$

$$\boxed{R_{22}^{(1)} = a^* b + \varepsilon}$$

Step 2: $R_{ij}^{(2)}$ — Intermediate states allowed: $\{1, 2\}$

Using the recurrence with $k = 2$:

$$R_{ij}^{(2)} = R_{i2}^{(1)} \left(R_{22}^{(1)}\right)^* R_{2j}^{(1)} + R_{ij}^{(1)}$$

Note $(R_{22}^{(1)})^* = (a^* b + \varepsilon)^* = (a^* b)^*$.

$\boldsymbol{R_{11}^{(2)}}$ (needed for the final answer):

$$R_{11}^{(2)} = R_{12}^{(1)}(R_{22}^{(1)})^* R_{21}^{(1)} + R_{11}^{(1)} = a^* b \cdot (a^* b)^* \cdot a^+ + a^* = a^* b (a^* b)^* a^+ + a^* = a^* (a^* b)^+ a^+ + a^*$$

We can simplify $a^* b (a^* b)^* = a^* (b a^*)^* b$ (rewriting the iteration):

$$\boxed{R_{11}^{(2)} = a^* \left(b a^+\right)^*}$$

Verification of the simplification: $a^* b (a^* b)^* a^+ + a^*$. Each term $b(a^* b)^* a^+$ represents one or more b-blocks each followed by one or more a's. Factoring out the leading a^*:

$$a^* b (a^* b)^* a^+ + a^* = a^* \left(b (a^* b)^* a^+ + \varepsilon\right) = a^* \left(b a^+\right)^* \text{ (by Arden's identity)}$$

The remaining expressions for completeness:

$\boldsymbol{R_{12}^{(2)}}$:

$$R_{12}^{(2)} = R_{12}^{(1)}(R_{22}^{(1)})^* R_{22}^{(1)} + R_{12}^{(1)} = a^* b (a^* b)^* (a^* b + \varepsilon) + a^* b = a^* b (a^* b)^* + a^* b = a^* (b a^*)^* b$$

$\boldsymbol{R_{21}^{(2)}}$:

$$R_{21}^{(2)} = R_{22}^{(1)}(R_{22}^{(1)})^* R_{21}^{(1)} + R_{21}^{(1)} = (a^* b + \varepsilon)(a^* b)^* a^+ + a^+ = (a^* b)^* a^+ + a^+ = (a^* b)^* a^+$$

$\boldsymbol{R_{22}^{(2)}}$:

$$R_{22}^{(2)} = R_{22}^{(1)}(R_{22}^{(1)})^* R_{22}^{(1)} + R_{22}^{(1)} = (a^* b)^* (a^* b + \varepsilon) + (a^* b + \varepsilon) = (a^* b)^* + \varepsilon = (a^* b)^*$$

Resulting Regular Expression for $L(\mathcal{A})$

The language $L(\mathcal{A})$ is accepted starting from the start state $p = 1$ and ending in any accept state. Since the only accept state is $p = 1$, we need all strings that start and end in state 1, using any intermediate states $\{1, 2\}$. This is given by $R_{11}^{(2)}$:

$$\boxed{R \;=\; R_{11}^{(2)} \;=\; a^* \left(b a^+\right)^*}$$

Interpretation

The language $L(R) = L(\mathcal{A})$ consists of all strings over $\{a, b\}$ that:

- start with zero or more a's, then
- repeat zero or more times the pattern: one b followed by one or more a's.

In particular, every b in an accepted string must be followed by at least one a. The string cannot end with b (since q is not an accept state). Examples: ε, a, aa, aba, $aaba$, $abba$ (rejected), bab (rejected).

Given DFA (Figure 4.15)

- **States:** $A, B, C, D, E, F, G, H, I$
- **Start state:** A
- **Accept states:** C, F, I

State	**0**	**1**
$\rightarrow A$	B	E
B	C	F
$*C$	D	H
D	E	H
E	F	I
$*F$	G	B
G	H	B
H	I	C
$*I$	A	E

Part (a): Table of Distinguishabilities

Method

Two states p and q are **distinguishable** if there exists some string w such that exactly one of $\delta(p, w)$ and $\delta(q, w)$ is an accepting state. We build the table iteratively:

1. **Base case:** Mark $\{p, q\}$ if exactly one is an accept state.
2. **Propagation:** Mark $\{p, q\}$ if for some input $x \in \{0, 1\}$, the pair $\{\delta(p, x), \delta(q, x)\}$ is already marked.
3. Repeat until no new pairs can be marked.

Base Case

Accept states $= \{C, F, I\}$; Non-accept states $= \{A, B, D, E, G, H\}$.

All pairs $\{p, q\}$ with one accept and one non-accept state are immediately marked distinguishable ($\times$).

Propagation

We check all remaining pairs (both accept, or both non-accept):

Non-accept pairs that become distinguishable:

$$\{A,B\} : \delta(A,0) = B,\ \delta(B,0) = C \Rightarrow \{B,C\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{A,E\} : \delta(A,0) = B,\ \delta(E,0) = F \Rightarrow \{B,F\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{A,H\} : \delta(A,0) = B,\ \delta(H,0) = I \Rightarrow \{B,I\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{B,D\} : \delta(B,0) = C,\ \delta(D,0) = E \Rightarrow \{C,E\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{B,G\} : \delta(B,0) = C,\ \delta(G,0) = H \Rightarrow \{C,H\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{D,E\} : \delta(D,0) = E,\ \delta(E,0) = F \Rightarrow \{E,F\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{D,H\} : \delta(D,0) = E,\ \delta(H,0) = I \Rightarrow \{E,I\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{E,G\} : \delta(E,0) = F,\ \delta(G,0) = H \Rightarrow \{F,H\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$
$$\{G,H\} : \delta(G,0) = H,\ \delta(H,0) = I \Rightarrow \{H,I\} \text{ marked} \Rightarrow \textbf{marked } (\times)$$

Pairs that remain indistinguishable (no marking possible):

$$\{A,D\} : \delta(\cdot,0) \to \{B,E\},\ \delta(\cdot,1) \to \{E,H\} \quad \text{(both unmarked)}$$
$$\{A,G\} : \delta(\cdot,0) \to \{B,H\},\ \delta(\cdot,1) \to \{E,B\} \quad \text{(both unmarked)}$$
$$\{D,G\} : \delta(\cdot,0) \to \{E,H\},\ \delta(\cdot,1) \to \{H,B\} \quad \text{(both unmarked)}$$
$$\{B,E\} : \delta(\cdot,0) \to \{C,F\},\ \delta(\cdot,1) \to \{F,I\} \quad \text{(both accept pairs, unmarked)}$$
$$\{B,H\} : \delta(\cdot,0) \to \{C,I\},\ \delta(\cdot,1) \to \{F,C\} \quad \text{(both accept pairs, unmarked)}$$
$$\{E,H\} : \delta(\cdot,0) \to \{F,I\},\ \delta(\cdot,1) \to \{I,C\} \quad \text{(both accept pairs, unmarked)}$$
$$\{C,F\} : \delta(\cdot,0) \to \{D,G\},\ \delta(\cdot,1) \to \{H,B\} \quad \text{(both unmarked)}$$
$$\{C,I\} : \delta(\cdot,0) \to \{D,A\},\ \delta(\cdot,1) \to \{H,E\} \quad \text{(both unmarked)}$$
$$\{F,I\} : \delta(\cdot,0) \to \{G,A\},\ \delta(\cdot,1) \to \{B,E\} \quad \text{(both unmarked)}$$

Complete Distinguishability Table

In the table below, $\times$ denotes a distinguishable pair and $\equiv$ denotes an indistinguishable (equivalent) pair. Pairs that are trivially marked (one accept, one non-accept) are shown as $\times$.

	A	B	C	D	E	F	G	H
B	$\times$							
C	$\times$	$\times$						
D	$\equiv$	$\times$	$\times$					
E	$\times$	$\equiv$	$\times$	$\times$				
F	$\times$	$\times$	$\equiv$	$\times$	$\times$			
G	$\equiv$	$\times$	$\times$	$\equiv$	$\times$	$\times$		
H	$\times$	$\equiv$	$\times$	$\times$	$\equiv$	$\times$	$\times$	
I	$\times$	$\times$	$\equiv$	$\times$	$\times$	$\equiv$	$\times$	$\times$

Part (b): Minimum-State Equivalent DFA

Equivalence Classes

The indistinguishable pairs yield three equivalence classes:

New State	Original States	Role
$[A]$	$\{A,\ D,\ G\}$	Start state
$[B]$	$\{B,\ E,\ H\}$	Non-accept
$[C]$	$\{C,\ F,\ I\}$	Accept state

Transition Table of Minimized DFA

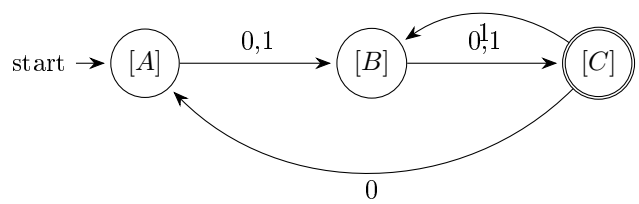
Using representative states to compute transitions:

State	0	1
$\to [A]$	$\delta(A,0) = B \to [B]$	$\delta(A,1) = E \to [B]$
$[B]$	$\delta(B,0) = C \to [C]$	$\delta(B,1) = F \to [C]$
$*[C]$	$\delta(C,0) = D \to [A]$	$\delta(C,1) = H \to [B]$

Written more compactly:

State	0	1
$\to [A]$	$[B]$	$[B]$
$[B]$	$[C]$	$[C]$
$*[C]$	$[A]$	$[B]$

Diagram of Minimized DFA



Summary

The original 9-state DFA is reduced to a **3-state** minimum DFA. The minimized automaton accepts strings whose length is congruent to 2 (mod 3), i.e. strings of length $2, 5, 8, 11, \ldots$

Week 4 Discord Question

When minimizing a DFA, does it matter which accepting state you use as a representative for an equivalence class when computing the transitions of the minimized DFA?

2.5 Week 5

Show that the following languages are not regular using the pumping lemma:

$$L_1 = \{a^n b^m \in \{a,b\}^* \mid n, m \in \mathbb{N},\ n > m \geq 0\}$$
$$L_2 = \{a^{n^2} \in \{a\}^* \mid n \in \mathbb{N},\ n \geq 1\}$$
$$L_3 = \{a^p \in \{a\}^* \mid p \in \mathbb{N},\ p \text{ prime}\}$$
$$L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}$$

$L_1 = \{a^n b^m \mid n > m \geq 0\}$

Assume L_1 is regular, so it satisfies the pumping lemma with constant n.

Choose $w = a^{n+1}b^n \in L_1$ (since $n+1 > n \geq 0$), with $|w| = 2n+1 \geq n$. ✓

Decompose $w = xyz$. By condition 2, $|xy| \leq n < n+1$, so xy lies entirely within the a^{n+1} prefix. Hence $x = a^i$, $y = a^j$ with $j \geq 1$ (condition 1), and $z = a^{n+1-i-j}b^n$.

Pump down with $k = 0$:

$$w' = xz = a^{n+1-j}b^n.$$

Since $j \geq 1$, we have $|w'|_a = n+1-j \leq n = |w'|_b$, so $w' \notin L_1$.

Contradiction. □

$L_2 = \{a^{n^2} \mid n \geq 1\}$

Assume L_2 is regular with pumping constant n.

Choose $w = a^{n^2} \in L_2$, with $|w| = n^2 \geq n$. ✓

Decompose $w = xyz$. Since the alphabet is $\{a\}$, we have $x = a^i$, $y = a^j$, $z = a^{n^2-i-j}$ with $j \geq 1$ (condition 1) and $i + j \leq n$ (condition 2).

Pump with $k = 2$:

$$w' = xy^2z = a^{n^2+j}.$$

For $w' \in L_2$ we would need $n^2 + j = m^2$ for some integer $m \geq 1$. However, since $1 \leq j \leq n$:

$$n^2 < n^2 + j \leq n^2 + n < n^2 + 2n + 1 = (n+1)^2.$$

Thus $n^2 + j$ lies strictly between two consecutive perfect squares and is therefore not a perfect square, giving $w' \notin L_2$.

Contradiction. □

$L_3 = \{a^p \mid p \textbf{ prime}\}$

Assume L_3 is regular with pumping constant n.

Choose a prime $p \geq n+2$ (such a prime exists since there are infinitely many primes), and let $w = a^p \in L_3$, so $|w| = p \geq n$. ✓

Decompose $w = xyz$ with $x = a^i$, $y = a^j$ ($j \geq 1$, $i + j \leq n$), $z = a^{p-i-j}$. Set $s := i + j$ so that $1 \leq s \leq n$.

Pump with $k = p - j$. Note $k \geq 1$ because $j \leq n \leq p - 2 < p$. The pumped word has length:

$$|xy^{p-j}z| = i + (p-j)\cdot j + (p-i-j) = p + j(p-j-1) = p + pj - j^2 - j = p(1+j) - j(j+1) = (p-j)(1+j).$$

Since $p - j \geq p - n \geq 2$ and $j + 1 \geq 2$, the length $(p-j)(1+j)$ is a product of two integers both at least 2, hence *composite*. Therefore $xy^{p-j}z \notin L_3$.

Contradiction. □

$L_4 = \{a^k b^m a^{k+m} \mid k, m \geq 0\}$

Assume L_4 is regular with pumping constant n.

Choose $w = a^n b^n a^{2n} \in L_4$ (setting $k = m = n$), with $|w| = 4n \geq n$. ✓

Decompose $w = xyz$. By condition 2, $|xy| \leq n$, so xy lies entirely within the initial a^n block. Hence $x = a^i$, $y = a^j$ with $j \geq 1$, and $z = a^{n-i-j} b^n a^{2n}$.

Pump with $k = 2$:

$$w' = xy^2z = a^{n+j}\, b^n\, a^{2n}.$$

For $w' \in L_4$ we would need $w' = a^{k'} b^{m'} a^{k'+m'}$ for some $k', m' \geq 0$. Reading the structure of w': $k' = n + j$ and $m' = n$, so the trailing a-block must have length

$$k' + m' = (n + j) + n = 2n + j.$$

But the trailing block of w' has length $2n$. Since $j \geq 1$, we have $2n < 2n + j$, so $w' \notin L_4$.

Contradiction. □

Week 5 Discord Question

Is it always possible to find such a "clever" k, or are there non-regular languages where no single pumping value works and you need a different proof strategy entirely?

2.6 Week 6

Exercise A Solutions

1. TM for $L = \{10^n : n \in \mathbb{N}\}$ that outputs 10^{n+1}

Idea: Scan to the right over all symbols until the first blank is found, then write an additional 0.

Turing Machine:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where:

- $Q = \{q_0, q_1, q_{accept}\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, B\}$
- $F = \{q_{accept}\}$

Transition function:

$$\delta(q_0, 1) = (q_1, 1, R)$$
$$\delta(q_1, 0) = (q_1, 0, R)$$
$$\delta(q_1, B) = (q_{accept}, 0, R)$$

Example: Input 100 → Output 1000

2. TM for $L = \{10^n : n \in \mathbb{N}\}$ that outputs 1

Idea: Keep the leading 1 and erase all trailing 0's.

Turing Machine:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where:

- $Q = \{q_0, q_1, q_{accept}\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, B\}$
- $F = \{q_{accept}\}$

Transition function:

$$\begin{aligned} \delta(q_0, 1) &= (q_1, 1, R) \\ \delta(q_1, 0) &= (q_1, B, R) \\ \delta(q_1, B) &= (q_{accept}, B, R) \end{aligned}$$

Example: Input 1000 $\rightarrow$ Output 1

3. TM for all binary strings that swaps 0 and 1

Idea: Traverse the tape and replace each 0 with 1 and each 1 with 0.

Turing Machine:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

where:

- $Q = \{q_0, q_{accept}\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, 1, B\}$
- $F = \{q_{accept}\}$

Transition function:

$$\begin{aligned} \delta(q_0, 0) &= (q_0, 1, R) \\ \delta(q_0, 1) &= (q_0, 0, R) \\ \delta(q_0, B) &= (q_{accept}, B, R) \end{aligned}$$

Example: Input 1010 $\rightarrow$ Output 0101

Exercise 1: Decidability and r.e. Languages

We analyze each statement:

1. If L_1 and L_2 are decidable, then so is $L_1 \cup L_2$.

True.

Argument: Since L_1 and L_2 are decidable, there exist Turing machines M_1 and M_2 that halt on all inputs. Construct a machine M that on input w:

- Runs M_1 on w.
- Runs M_2 on w.
- Accepts if either accepts; otherwise rejects.

Since both machines halt, M halts, so $L_1 \cup L_2$ is decidable.

2. If L is decidable, then so is $\overline{L} = \Sigma^* \setminus L$.

True.

Argument: If M decides L, construct M' that:

- Runs M on input w.
- If M accepts, reject.
- If M rejects, accept.

Thus, M' decides $\overline{L}$.

3. If L is decidable, then so is L^*.

True.

Argument: Given input w, check all possible ways to split w into substrings

$$w = w_1 w_2 \cdots w_k$$

and verify whether each $w_i \in L$. Since L is decidable, each check halts. There are finitely many splits, so we can test all of them and decide whether $w \in L^*$.

4. If L_1 and L_2 are r.e., then $L_1 \cup L_2$ is r.e.

True.

Argument: Let M_1 and M_2 recognize L_1 and L_2. Construct a machine that:

- Simulates both M_1 and M_2 in parallel (dovetailing).
- Accepts if either machine accepts.

Thus, $L_1 \cup L_2$ is recursively enumerable.

5. If L is r.e., then so is $\overline{L}$.

False.

Counterexample: Let L be the Halting Problem:

$$L = \{\langle M, w \rangle : M \text{ halts on input } w\}$$

L is r.e., but $\overline{L}$ (non-halting pairs) is not r.e. Thus, the statement is false.

6. If L is r.e., then so is L^*.

True.

Argument: Given input w, enumerate all ways to split w into substrings and simulate the recognizer for L on each substring in parallel. If a decomposition exists where all substrings are accepted, then accept. Thus, L^* is r.e.

Summary:

- (1) True
- (2) True
- (3) True
- (4) True
- (5) False
- (6) True

Week 6 Discord Question

Why does giving a Turing machine the ability to loop forever increase its computational power compared to DFAs?

2.7 Week 7

Problems on Decidability Exercise 2: Solution

We classify each language as decidable, recursively enumerable (r.e.), or co-recursively enumerable (co-r.e.).

1. $L_1 = \{M \mid M$ halts on itself$\}$

Claim: L_1 is recursively enumerable but not decidable and not co-r.e.

Proof: To show L_1 is recursively enumerable, we construct a Turing machine that simulates M on input M. If M halts on input M, the simulation halts and accepts. If M does not halt, the simulation runs forever. Hence, L_1 is r.e.

To show L_1 is not decidable, note that this is a special case of the Halting Problem, which is undecidable.

To show L_1 is not co-r.e., observe that if both L_1 and its complement were recursively enumerable, then L_1 would be decidable, which is a contradiction.

□

2. $L_2 = \{(M, w) \mid M$ halts on $w\}$

Claim: L_2 is recursively enumerable but not decidable and not co-r.e.

Proof: To show L_2 is recursively enumerable, simulate M on input w. If M halts, accept; otherwise, run forever. Thus, L_2 is r.e.

L_2 is exactly the Halting Problem, which is known to be undecidable.

If L_2 were also co-r.e., then it would be decidable. Since it is not decidable, it cannot be co-r.e.

□

3. $L_3 = \{(M, w, k) \mid M$ halts on w in at most k steps$\}$

Claim: L_3 is decidable (and hence both r.e. and co-r.e.).

Proof: We construct a decider as follows: on input (M, w, k), simulate M on input w for at most k steps. If M halts within k steps, accept; otherwise, reject.

Since this procedure always halts after at most k steps, L_3 is decidable. Therefore, it is also both recursively enumerable and co-recursively enumerable.

□

Summary

Language	Decidable	r.e.	co-r.e.
L_1	No	Yes	No
L_2	No	Yes	No
L_3	Yes	Yes	Yes

Landau Notation: Solutions to Exercises 2-5

Exercise 2

Let $f, g, h : \mathbb{N} \to \mathbb{R}_{\geq 0}$.

1. $f \in \mathcal{O}(f)$

   Choose $M = 1$ and any $N \in \mathbb{N}$. Then for all $n \geq N$,

   $$f(n) \leq 1 \cdot f(n),$$

   hence $f \in \mathcal{O}(f)$.

2. **If $c > 0$, then $\mathcal{O}(cf) = \mathcal{O}(f)$**

   ($\subseteq$) Suppose $h \in \mathcal{O}(cf)$. Then there exist M, N such that

   $$h(n) \leq M \cdot cf(n) = (Mc)f(n),$$

   so $h \in \mathcal{O}(f)$.

   ($\supseteq$) Similarly, if $h \in \mathcal{O}(f)$, then

   $$h(n) \leq Mf(n) = M \cdot \frac{1}{c}(cf(n)),$$

   so $h \in \mathcal{O}(cf)$.

3. **If $f \leq g$ eventually, then $\mathcal{O}(f) \subseteq \mathcal{O}(g)$**

   If $f(n) \leq g(n)$ for all $n \geq N_0$, and $h \in \mathcal{O}(f)$, then

   $$h(n) \leq Mf(n) \leq Mg(n),$$

   for all sufficiently large n. Hence $h \in \mathcal{O}(g)$.

4. **If $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(f + h) \subseteq \mathcal{O}(g + h)$**

   Let $k \in \mathcal{O}(f + h)$, so

   $$k(n) \leq M(f(n) + h(n)).$$

Since $f \in \mathcal{O}(g)$, we have $f(n) \leq M'g(n)$ for large n. Thus,

$$k(n) \leq M(M'g(n) + h(n)) \leq M''(g(n) + h(n)),$$

hence $k \in \mathcal{O}(g + h)$.

5. **If $h(n) > 0$ and $\mathcal{O}(f) \subseteq \mathcal{O}(g)$, then $\mathcal{O}(fh) \subseteq \mathcal{O}(gh)$**

   Let $k \in \mathcal{O}(fh)$. Then

$$k(n) \leq Mf(n)h(n).$$

   Since $f(n) \leq M'g(n)$ for large n,

$$k(n) \leq MM'g(n)h(n),$$

   hence $k \in \mathcal{O}(gh)$.

Exercise 3

Let $i, j, k, n \in \mathbb{N}$.

1. If $j \leq k$, then $n^j \leq n^k$ for $n \geq 1$, so

$$\mathcal{O}(n^j) \subseteq \mathcal{O}(n^k).$$

2. If $j \leq k$, then

$$n^j + n^k \leq 2n^k,$$

   hence

$$\mathcal{O}(n^j + n^k) \subseteq \mathcal{O}(n^k).$$

3. For a polynomial,

$$\sum_{i=0}^{k} a_i n^i \leq Cn^k$$

   for some constant C, so

$$\mathcal{O}\left(\sum_{i=0}^{k} a_i n^i\right) = \mathcal{O}(n^k).$$

4. For $n \geq 2$, $\log n \leq n$, so

$$\mathcal{O}(\log n) \subseteq \mathcal{O}(n).$$

5. Since $\log n \leq n$,

$$n \log n \leq n^2,$$

   hence

$$\mathcal{O}(n \log n) \subseteq \mathcal{O}(n^2).$$

Exercise 4

1. $\mathcal{O}(\sqrt{n}) \subseteq \mathcal{O}(n)$ since $\sqrt{n} \leq n$.
2. $\mathcal{O}(n^2) \subseteq \mathcal{O}(2^n)$ since exponentials dominate polynomials.
3. $\mathcal{O}(\log n) \subseteq \mathcal{O}((\log n)^2)$ since $\log n \leq (\log n)^2$ for large n.
4. $\mathcal{O}(2^n) \subseteq \mathcal{O}(3^n)$ since $2^n \leq 3^n$.
5. $\mathcal{O}(\log_2 n) = \mathcal{O}(\log_3 n)$ since logarithms differ by a constant factor:

$$\log_2 n = \frac{\log_3 n}{\log_3 2}.$$

Exercise 5

We compare runtimes of sorting algorithms:

- **Bubble Sort vs. Insertion Sort:** Both have worst-case runtime $\mathcal{O}(n^2)$. However, insertion sort performs better on nearly sorted data ($\mathcal{O}(n)$ best case), while bubble sort generally remains inefficient.
- **Insertion Sort vs. Merge Sort:** Insertion sort is $\mathcal{O}(n^2)$ in the worst case, while merge sort is $\mathcal{O}(n \log n)$ in all cases. Thus, merge sort is asymptotically more efficient.
- **Merge Sort vs. Quick Sort:** Both have average runtime $\mathcal{O}(n \log n)$. However:
  - Merge sort guarantees $\mathcal{O}(n \log n)$.
  - Quick sort has worst-case $\mathcal{O}(n^2)$ but is often faster in practice due to smaller constants and cache efficiency.

Conclusion: Asymptotically, algorithms with $\mathcal{O}(n \log n)$ dominate $\mathcal{O}(n^2)$ algorithms, while practical performance may still depend on constants and input characteristics.

Week 6 Discord Question

Big O ignores constant factors and lower-order terms, but in real implementations those can still affect performance, so how do we decide when two algorithms in the same Big O class are actually meaningfully different in practice?

2.8 Week 8

Exercise 1

1.

$$\varphi_1 := \neg((a \wedge b) \vee (\neg c \wedge d))$$

$$\begin{aligned}\varphi_1 &= \neg(a \wedge b) \wedge \neg(\neg c \wedge d) \\ &= (\neg a \vee \neg b) \wedge (c \vee \neg d)\end{aligned}$$

CNF:

$$(\neg a \vee \neg b) \wedge (c \vee \neg d)$$

2.

$$\varphi_2 := \neg((p \vee q) \to (r \wedge \neg s))$$

$$\begin{aligned}\varphi_2 &= \neg(\neg(p \vee q) \vee (r \wedge \neg s)) \\ &= \neg(\neg(p \vee q)) \wedge \neg(r \wedge \neg s) \\ &= (p \vee q) \wedge (\neg r \vee s)\end{aligned}$$

CNF:

$$(p \vee q) \wedge (\neg r \vee s)$$

Exercise 2

1.

$$\psi_1 := (a \vee \neg b) \wedge (\neg a \vee b) \wedge (\neg a \vee \neg b)$$

Try assignment: $a = \text{false},\ b = \text{false}$

$$(a \vee \neg b) = F \vee T = T$$
$$(\neg a \vee b) = T \vee F = T$$
$$(\neg a \vee \neg b) = T \vee T = T$$

Conclusion: Satisfiable.

2.

$$\psi_2 := (\neg p \vee q) \wedge (\neg q \vee r) \wedge \neg(\neg p \vee r)$$

$$\neg(\neg p \vee r) = p \wedge \neg r$$

So:

$$= (\neg p \vee q) \wedge (\neg q \vee r) \wedge p \wedge \neg r$$

From $p \wedge \neg r$:

$$p = T, \quad r = F$$

Substitute:

$$(\neg p \vee q) = F \vee q = q$$
$$(\neg q \vee r) = \neg q \vee F = \neg q$$

$$q \wedge \neg q$$

Conclusion: Not satisfiable.

3.

$$\psi_3 := (x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$$

From the clauses:

$$(x \vee y) \wedge (\neg x \vee y) \Rightarrow y$$
$$(x \vee \neg y) \wedge (\neg x \vee \neg y) \Rightarrow \neg y$$

$$y \wedge \neg y$$

Conclusion: Not satisfiable.

Week 6 Discord Question

When simplifying a Boolean formula to check satisfiability, sometimes certain variable assignments seem "forced" by the clauses (like setting a variable to true or false to satisfy multiple clauses at once). Is there a systematic way to identify these forced assignments early, and how does this relate to algorithms like unit propagation?

3 Synthesis

4 Evidence of Participation

5 Conclusion

References

[BLA] Author, Title, Publisher, Year.